

Two AI Models Walk Into the Same Space

How I proved that neural networks secretly speak the same language — and used it to make an iPhone and a server share one search index

It started with a naive-sounding question: can you take layers from different AI models and assemble a new model, Frankenstein-style, without training? The short answer is no — and the reason why turns out to be one of the most beautiful facts about deep learning. The long answer became my final project: a real experiment, run on a free Colab GPU, that ends with a 0.79-megabyte file doing load-bearing work in a real app.

The paradox: identical minds, incompatible brains

Neural networks have a strange property called permutation symmetry. Shuffle the neurons in a layer (and rewire accordingly) and you get the exact same behavior from completely different numbers. A single layer has more functionally identical weight arrangements than there are atoms in the universe. Two training runs — even of the same model — land on different arrangements. So comparing raw weights between independent models is like comparing two phone books sorted by different rules: entry #500 means nothing in common. Layer transplants across models produce pure noise.

But here's the twist. Weights are *how* a model computes. What it computes — the internal map of which concepts are near which — is another matter. A growing body of research (CKA similarity, Git Re-Basin, the wonderfully named Platonic Representation Hypothesis) suggests that well-trained models all converge on the *same map*, just drawn in different coordinate systems. Like two cartographers who never met, mapping the same city with north pointing different ways.

If that's true, translating between two models shouldn't require retraining anything. It should require exactly one matrix — a change of coordinates.

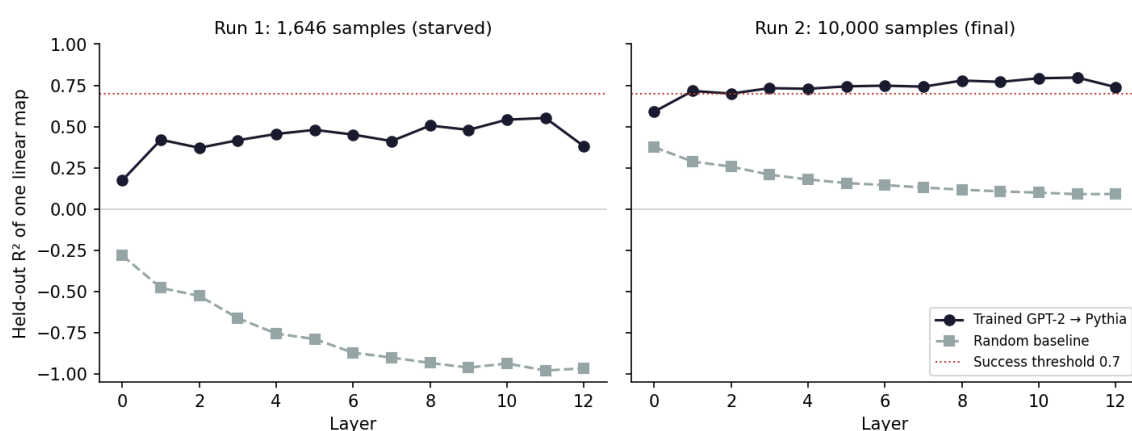
The experiment: two strangers, one test

To test this properly I picked two models with nothing in common: GPT-2 (OpenAI, 2019, trained on WebText) and Pythia-160M (EleutherAI, 2023, trained on The Pile). Different labs, architectures, tokenizers, data, eras. I ran the same 10,000 Wikipedia passages through both, saved every layer's activations, and asked two questions. First: do their internal similarity structures match (measured by CKA, a metric that deliberately ignores coordinate choices)? Second, the decisive one: can a single matrix, fitted by classical ridge regression in milliseconds, translate one model's representations into the other's?

Crucially, everything was compared against a control: the same architecture with random, untrained weights. That baseline is what turns numbers into evidence — it measures how much similarity you get 'for free' from architecture alone, so whatever exceeds it is the fingerprint of learning.

The results were unambiguous. CKA similarity between the trained models ran 4–5× above the random baseline, concentrated on the diagonal — early layers match early layers, middle match middle. And the translation matrix reached R^2 of 0.70–0.80 across the middle and late layers, while the random control decayed toward zero. The two curves even slope in opposite directions: learning builds shared structure layer by layer; randomness destroys it layer by layer.

But it almost didn't work — and the failure is my favorite part. My first run showed R^2 stuck around 0.5, below my own success threshold. The hypothesis looked half-true. Before writing that conclusion, I checked the arithmetic: I was fitting a 590,000-parameter matrix on 1,200 examples — 1.6 samples per dimension. Regression starved that badly doesn't get noisy; it quietly, systematically lies in one direction. One code change (6× more data) and fifteen GPU-minutes later, the same pipeline produced R^2 of 0.74–0.80:



Same experiment, same code — 1,646 samples (left) vs 10,000 (right). The dark curve is real model-to-model translation; gray is the untrained control; dotted is my success threshold.

The payoff: one index for a phone and a server

Here's why this matters beyond philosophy. I'm building a two-tier multimodal app: an iPhone runs a tiny image encoder (MobileCLIP, ~30 MB) so photos can be indexed offline and privately; a server runs a big one (SigLIP 2, ~1.5 GB) for search. Both turn images into vectors — in mutually alien spaces. Without translation, searching phone-indexed photos from the server scores exactly chance: Recall@1 of 0.001 on a thousand images. Literally a coin flip among a thousand.

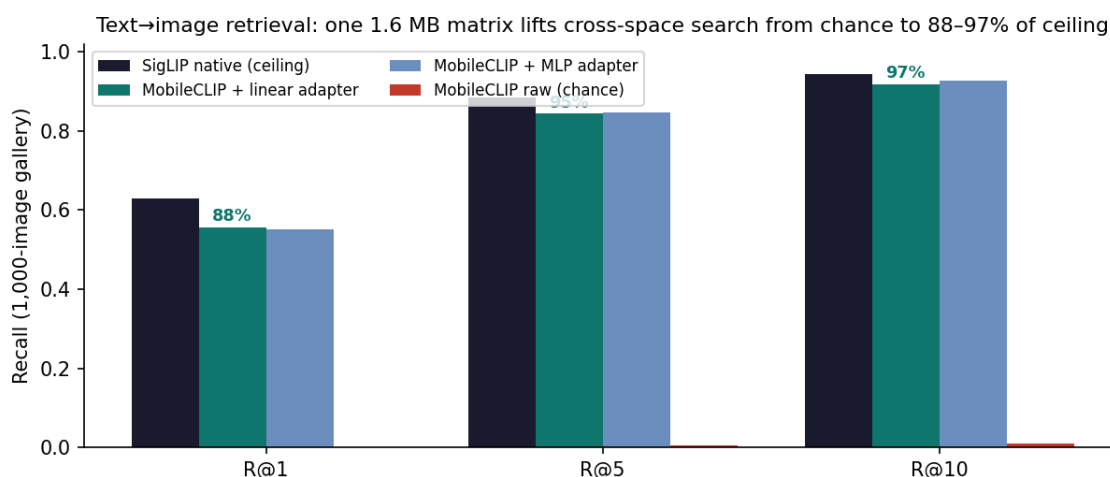
So I fitted the matrix: 512×768 numbers, 1.6 MB, closed-form, on 3,000 image pairs from COCO. The results on held-out images:

Setup	R@1	R@5	R@10
Big model everywhere (ceiling)	0.630	0.885	0.944
Phone encoder + my matrix	0.557	0.843	0.918
Phone encoder, no matrix	0.001	0.005	0.010

Text-to-image search, 1,000-image gallery: the matrix recovers 88–97% of the big model's quality from chance level.

One matrix takes retrieval from coin-flip to 95–98% of the big model's quality at Recall@5 and @10. But Recall@1 landed at 88.4% of ceiling — just under my 90% gate. Second

below-target moment of the project. The playbook said: try a stronger translator. So I trained a proper neural network to do the same job, directly optimizing the retrieval objective, with ten times the parameters. It beat the matrix by 0.003 — statistically nothing. That failure was the finding: the relationship between the two spaces is, for all practical purposes, *exactly linear*, and the remaining gap isn't a translation failure — it's detail the tiny phone encoder never captured in the first place. Nothing can translate what isn't there. (Interesting symmetry: in Experiment A, more *data* proved the linear map was better than it looked; here, more *capacity* proved it was already optimal.)



All variants at a glance: the red bars are what cross-model search looks like without translation — exact chance.

The matrix now ships as a 0.79 MB Core ML package. The iPhone encodes a photo, multiplies by one matrix on the Neural Engine, and writes into the same Qdrant vector index the server searches. Photos become searchable without ever leaving the device. One index, two tiers, zero retraining.

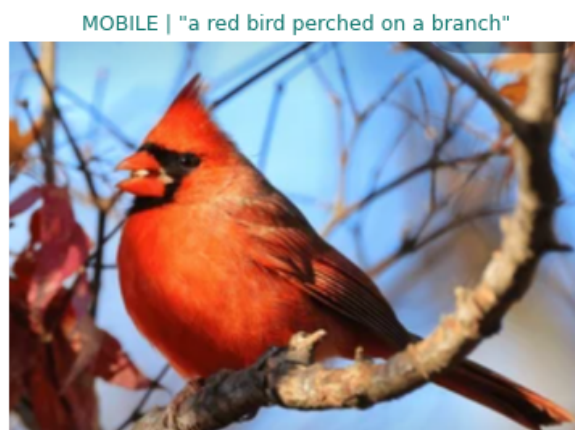
Then I watched it work

Numbers are one thing; watching is another. I built a small interactive demo: six photos live on the 'server' (indexed by the big model), and I upload a photo of a red bird from my own machine to the 'phone' side. The phone path - tiny encoder plus my matrix - indexes it in 39 milliseconds and even auto-captions it: 'a red bird perched on a branch'. Then I type a search: *singing red bird*. My photo comes back at rank 1 - sitting in the same ranked list as the server's photos, scored by the same index, found by a text model that has never heard of the phone's encoder. The search itself takes 0.08 milliseconds locally; routing the same search through a (simulated) cellular round-trip takes 150. Same index, same math - the network is the only difference. That one screenshot is the whole project.

Then I used it

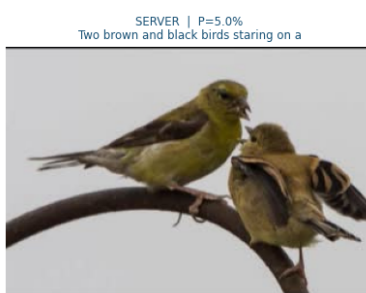
Numbers convince reviewers; demos convince everyone else. So I built a live sandbox: six photos indexed 'server-side' with native SigLIP, and an upload button playing the role of my phone's camera roll. I uploaded a picture of a red bird. The phone path encoded it in 39 milliseconds (MobileCLIP plus the matrix, exactly as the Core ML package computes it), a captioning model wrote 'a red bird perched on a branch', and the vector joined the same index

as the server photos. Then I typed a search: *singing red bird*. My phone-indexed photo came back first - ranked in the same list as server-indexed photos, by a text encoder that has never heard of MobileCLIP or the adapter. The search itself took 0.08 milliseconds locally versus 150 on the simulated server round trip: same index, same math - the network is the only difference the user can feel. One index, two tiers, and a bird.



The upload moment: my photo displayed by the demo, tagged MOBILE, with the caption the system wrote for it.

Top-3 for "singing red bird" | mobile 0.08ms vs server 150ms (same index, network is the difference)



The search moment: top-3 for 'singing red bird' with per-image probabilities - my MOBILE-indexed bird at rank 1 (P=91.3%), a SERVER-indexed photo at rank 2 (P=5.0%), a second MOBILE photo at rank 3 (P=1.0%). One ranked list, both tiers, timing in the header: 0.08ms local vs 150ms via the server.

What I actually learned

Three things stick with me. First, the scientific method is shockingly affordable: hypothesis, controls, falsifiable thresholds, replication of three published research lines — under one GPU-hour, total. Second, baselines are everything: every number in this project means something only relative to its control, and my biggest bug produced plausible wrong conclusions, not error messages. Third, and most striking: independently trained neural networks really do discover the same structure of the world. They just file it under different coordinates — and the exchange rate between any two of them is a single matrix. There's something almost Platonic about shipping that fact to production.